Since both applications depend on PostgreSQL, Docker Compose will also run the PostgreSQL container.

When the downstream services are all up and running, it's time to start Edge Service. From a Terminal window, navigate to the project's root folder (edge-service), and run the following command:

```
$ ./gradlew bootRun
```

The Edge Service application will start accepting requests on port 9000. For the final test, try executing operations on books and orders, but this time through the API gateway (that is, using port 9000 rather than the individual ports to which Catalog Service and Order Service are listening). They should return a `200 OK` response:

```
$ http :9000/books
$ http :9000/orders
```

The result is the same as if you called Catalog Service and Order Service directly, but you only need to know one hostname and port this time. When you are done testing the application, stop its execution with Ctrl-C. Then terminate all the containers with Docker Compose:

```
$ docker-compose down
```

Under the hood, Edge Service uses Netty's HTTP client to forward requests to downstream services. As extensively discussed in the previous chapter, whenever an application calls an external service, it's essential to configure a timeout to make it resilient to interprocess communication failures. Spring Cloud Gateway provides dedicated properties to configure the HTTP client timeouts.